



EMBEDDED SOFTWARE ENGINEERING

Session 5 – Event-Driven Programming

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last time: a menu built with OOP-style polymorphism, dispatched through `activate/adjust/draw`.
 - The app still *polls* the BSP every cycle
- Today: replace polling with events, understand why a queue and not a variable
- Everything lands in a real refactor: `td4` becomes `td5`, put LEDs in a sub-page

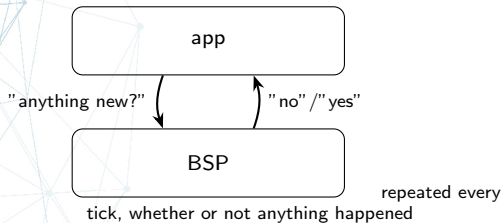
Today

1 The Event Queue

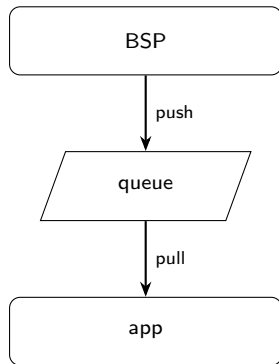
2 Event-driven refactor

Polling vs. event-driven

Polling (today)



Event-driven (today's goal)



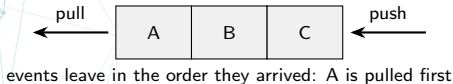
one-way: the app never asks, it drains what's waiting

Why not just... one variable?

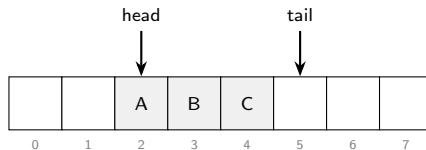
- Imagine the app stores the latest event in a single variable, `last_event`
- Three events arrive, in this order, before the app runs again: ROTATE, ROTATE, CLICK
- **One variable:** one box, overwritten each time
 - When the app finally looks, it only sees CLICK
 - Both ROTATEs are gone, never seen, and nothing tells us they existed
- **A queue:** as many slots as we choose
 - All three events are kept, in order, until the app drains them
- Nothing guarantees only one event happens between two superloop iterations
- A single “last event” slot silently drops whatever came before it

A queue: FIFO, implemented as a ring buffer

1. FIFO: first in, first out



2. Implementation: array + head + count



here $\text{head} = 2$, $\text{count} = 3$, so the next push goes to slot 5

- Push writes at $\text{tail} = (\text{head} + \text{count}) \% \text{CAPACITY}$, then $\text{count}++$
- Pull reads at head, then $\text{head} = (\text{head} + 1) \% \text{CAPACITY}$ and $\text{count}--$
- The modulo makes the indices wrap around: the array is a circle, nothing is ever moved or copied
- Full queue ($\text{count} == \text{CAPACITY}$): push returns false, the event is dropped

Pushing from an interrupt: dangerous, but we do it anyway

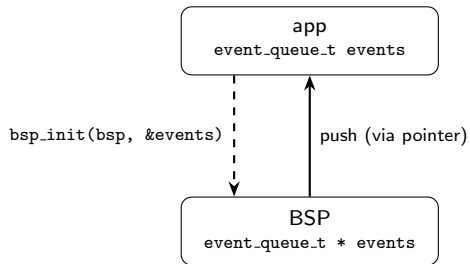
- Today the EXTI callback (an ISR) pushes button events directly, while `app_process` pulls from the main loop
- Why accept it?
 - No flag to poll, no latency: the event is in the queue the moment the button is pressed
 - It keeps the BSP → queue → app design simple, which is the point of this session
- Why is it dangerous? `push` and `pull` both modify shared state (`count`, `head`) with several non-atomic steps
 - If the ISR fires in the middle of a `pull`, it sees a half-updated queue: corrupted `count`, lost or duplicated event
 - `volatile` does not help: it prevents the compiler from caching a value, not the interruption in the middle of a read-modify-write

Looking ahead

We do not have the tools to fix this properly yet. Later, FreeRTOS's queues will do it for us: they are safe to use between ISRs and tasks (critical sections done inside the queue functions).

Who owns the queue?

- The queue belongs in the **app**: the app is the consumer, it decides what events mean
- But the **BSP** needs it to push events
- The BSP must not depend on the app (layering): so app gives it a **pointer** at init time
- The BSP only knows `event_queue_t`, not `app_t`



Queue interface & events

```
typedef enum {
    EVT_ENCODER_ROTATE, /* data : delta */
    EVT_ENCODER_CLICK,
    EVT_SW1_CLICK,
    EVT_SW2_CLICK,
} app_event_type_t;

typedef struct {
    app_event_type_t type;
    int32_t data;
} app_event_t;

#define EVENT_QUEUE_CAPACITY 16

typedef struct {
    app_event_t items[EVENT_QUEUE_CAPACITY];
    int head;
    int count;
} event_queue_t;

void event_queue_init(event_queue_t *q);
bool event_queue_push(event_queue_t *q, app_event_t evt);
bool event_queue_pull(event_queue_t *q, app_event_t *out);
```

Queue implementation

```
void event_queue_init(event_queue_t *q) {
    q->head = 0;
    q->count = 0;
}

bool event_queue_push(event_queue_t *q, app_event_t evt) {
    if (q->count >= EVENT_QUEUE_CAPACITY) {
        return false;
    }
    int tail = (q->head + q->count) % EVENT_QUEUE_CAPACITY;
    q->items[tail] = evt;
    q->count++;
    return true;
}

bool event_queue_pull(event_queue_t *q, app_event_t *out) {
    if (q->count == 0) {
        return false;
    }
    *out = q->items[q->head];
    q->head = (q->head + 1) % EVENT_QUEUE_CAPACITY;
    q->count--;
    return true;
}
```

Refactor : Give the event queue pointer to BSP

```
void bsp_init(bsp_t * bsp, event_queue_t * events);
```

- bsp_init now takes a pointer to the queue owned by the app
- The BSP stores it in a new event_queue_t * events field of its private context (bsp_ctx_t)

```
typedef struct bsp_ctx_struct {  
    event_queue_t * events;  
    /* ... */  
} bsp_ctx_t;
```

- The context is what the ISR callback and bsp_process use to reach the queue

Refactor : Push EVT_..._CLICK

```
void HAL_GPIO_EXTI_Callback(uint16_t pin) {  
    bsp_ctx_t * ctx = &bsp_ctx;  
  
    if (pin == BTN_SW1_Pin) {  
        event_queue_push(ctx->events, (app_event_t){.type = EVT_SW1_CLICK});  
    }  
    if (pin == BTN_SW2_Pin) {  
        event_queue_push(ctx->events, (app_event_t){.type = EVT_SW2_CLICK});  
    }  
    if (pin == ENCODER_PB_Pin) {  
        event_queue_push(ctx->events, (app_event_t){.type = EVT_ENCODER_CLICK});  
    }  
}
```

Refactor : Push EVT_ENCODER_ROTATE

```
void bsp_process(struct bsp_struct * bsp) {  
    bsp_ctx_t * ctx = (bsp_ctx_t *)bsp->ctx;  
  
    int32_t count = (int32_t)(int16_t)__HAL_TIM_GET_COUNTER(&htim3);  
    int32_t delta = count - ctx->encoder_last_count;  
    ctx->encoder_last_count = count;  
    if (delta != 0) {  
        event_queue_push(ctx->events, (app_event_t){.type = EVT_ENCODER_ROTATE, .data = delta});  
    }  
}
```

Refactor : The queue lives in app_t

- Add an `event_queue_t events` field to `app_t` (in `app.h`), next to the `bsp_t *` pointer and the menus
- Initialise it in `app_init`, before the BSP can push anything:

```
typedef struct {  
    bsp_t * bsp;  
    event_queue_t events;  
    /* ... */  
} app_t;  
  
void app_init(app_t *app, bsp_t *bsp) {  
    app->bsp = bsp;  
  
    event_queue_init(&app->events);  
    /* ... */  
}
```

- The BSP receives `&app->events` through `bsp_init`, as seen on the previous slides

Refactor : menu_handle_event (1/2)

```
menu_t *menu_handle_event(menu_t *m, app_event_t *evt, bsp_t *bsp) {  
    menu_entry_t *entry = m->entries[m->cursor];  
  
    if (m->editing) {  
        switch (evt->type) {  
            case EVT_ENCODER_ROTATE:  
                menu_entry_adjust(entry, evt->data);  
                return NULL;  
            case EVT_ENCODER_CLICK:  
                m->editing = false;  
                return NULL;  
            default:  
                return NULL;    // Ignore other events for now  
        }  
    }  
}
```

Refactor : menu_handle_event (2/2)

```
switch (evt->type) {  
    case EVT_ENCODER_ROTATE:  
        m->cursor += (int)evt->data;  
        if (m->cursor < 0) {  
            m->cursor = 0;  
        }  
        if (m->cursor >= m->count) {  
            m->cursor = m->count - 1;  
        }  
        entry = m->entries[m->cursor];  
  
        return NULL;  
    case EVT_ENCODER_CLICK:  
        if (entry->kind == MENU_ENTRY_HSV_COMPONENT) {  
            m->editing = true;  
        } else {  
            return menu_entry_activate(entry, bsp);  
        }  
        return NULL;  
    default:  
        return NULL;  
}
```

Refactor : app_process

```
void app_process(app_t *app) {  
    bsp_t * bsp = app->bsp;  
    app_event_t evt;  
  
    while (event_queue_pull(&app->events, &evt)) {  
        menu_t *next = menu_handle_event(app->active, &evt, bsp);  
  
        if (next != NULL) {  
            app->active = next;  
            menu_enter(app->active);  
        }  
  
        bsp->neopixel_set_hsv(bsp, app->hue, app->sat, app->val);  
  
        menu_draw(app->active, bsp);  
    }  
}
```

Take away

- Polling asks, event-driven receives – same hardware detection, different contract with the app
- A queue survives what a variable can't: more than one event between two passes of the superloop
- The queue belongs to the consumer, not the producer – and once it does, anyone can post into it
- Next session: state machines – naming the ifs we just wrote (blink, and the menu's own edit-mode) for what they really are

Exercise: a dedicated LEDS page

- Right now, the main page lists four rows: LED1, LED2, LED3, NEOPIXEL
- **Goal:** bring that down to two rows, LEDS and NEOPIXEL, each opening its own page

Specifications

- Main page: exactly two rows, LEDS and NEOPIXEL – both open a page, neither toggles anything itself
- LEDS page: the three LED rows, behaving exactly as they do today (click toggles), plus a BACK row at the end, back to the main page
- NEOPIXEL page: unchanged – HUE, SAT, VAL, BACK

Everything you need already exists in the project – nothing new to invent, just move entries to a different menu_t.

Target page structure

